

Abstract—Security Operations Centers (SOCs) evaluate their effectiveness with aggregate statistics – detection coverage, response-time distributions, and posture scores – yet most commercially deployed tooling computes these figures from data whose provenance is unverifiable. Dashboard panels that report “100% coverage” or “CRITICAL posture” are frequently unreproducible: the underlying counters may be hard-coded, computed over a window that excludes the relevant observations, or silently substituted with neutral defaults when a query returns no rows. This paper presents AI Sentinel, a real-time SOC platform whose central design commitment is verifiable ground truth: every metric the platform reports is derived by an auditable computation over persisted, tamper-evident records, and every quantity that cannot be established from real data is returned as an explicit null or a named absence state rather than a reassuring default.

The system ingests host, network, authentication, and file telemetry through a non-blocking asynchronous pipeline, applies seventeen runtime-configurable sliding-window detection rules and a drift-monitored Isolation Forest anomaly model, correlates detections into MITRE ATT&CK-mapped incidents, and streams results to an operator console over an authenticated WebSocket. We contribute four mechanisms: (i) an HMAC-SHA256 hash-chained audit ledger that makes administrative history tamper-evident, with verified-chain reporting; (ii) a **derived** security-posture score in which every point is an explained, bounded, drill-down delta, and where zero telemetry is scored as the worst possible state rather than as success; (iii) a **proof-carrying coverage** analysis that partitions MITRE techniques into VERIFIED_LIVE / DETECTING / RULES_DISABLED / OBSERVED_NO_RULES by joining rule definitions against observed telemetry; and (iv) a data-quality center that surfaces silent pipeline drops as first-class operational signals.

Evaluation on an 8,658-event live corpus shows an ingest-to-console p99 of 10.58 ms (100% of deliveries under the 2s operational target), a tamper-evident audit ledger of 2,847 records verifying intact, and an alert-compression ratio of 101.7 detection firings per analyst-facing alert. The evaluation process itself surfaced two silent pipeline defects – a `NameError` that silently dropped an entire malware detection class and a column allow-list that prevented incident timelines from accumulating – which we describe as evidence for the paper’s central claim: without a data-quality signal wired to a live counter, correctness failures of exactly this kind remain invisible to operators.

Index Terms—Security operations center, real-time security telemetry, MITRE ATT&CK, security posture assessment, tamper-evident logging, evidence integrity, unsupervised anomaly detection, data provenance, honest computing.

I. Introduction

A. Problem Statement

A Security Operations Center exists to answer three questions continuously: What is happening now?, can we prove what we did about it?, and how good are we at it?. Commercial SOC platforms answer the third question with dashboards. The methodological difficulty is that the measurements behind those dashboards are rarely themselves subjects of audit. A coverage percentage is usually computed as the ratio of enabled rules to configured rules – a statement about the configuration, not about whether any adversary would actually be detected. A posture score is usually a weighted sum of dashboard

sub-scores, where a component that fails to load quietly contributes its nominal default. A mean-time-to-detect figure is frequently averaged over a sample that excludes the very events that were missed.

The result is a measurement problem that compounds: analysts cannot trust the numbers, so they re-derive them by hand, so the automation saves little. Where honesty is attempted, it is usually attempted at the level of the display, while the underlying computation still defaults to benign values.

B. Contribution

This paper describes AI Sentinel, a real-time SOC platform built around a single invariant that we call verifiable ground truth: no displayed figure may be produced by a computation that cannot be traced to persisted records, and any quantity that real data cannot establish is reported as an explicit absence rather than a default. The invariant is enforced architecturally – it is a property of the storage layer, the scoring engines, and the API contract, not a documentation convention.

Our contributions are:

- 1) Evidence-grounded measurement architecture. A telemetry pipeline (Section III) in which normalization, detection, correlation, and delivery each persist a lifecycle timestamp, so that latency, detection, and response metrics are computed from observed state transitions rather than from counters maintained by the component being measured.
- 2) Tamper-evident audit ledger (Section IV-A). An HMAC-SHA256 hash chain over the administrative audit log, with legacy backfill and a verification endpoint that reports record counts, tampered record identifiers, and broken links.
- 3) Derived, explained posture scoring (Section IV-B). A posture engine in which the score is the sum of bounded per-factor deltas – each carrying a machine-readable cause – and in which absent telemetry is defined as the maximally adverse condition, because a security posture tool that reports “healthy” when it is blind is worse than no tool at all.
- 4) Proof-carrying MITRE coverage (Section IV-C). A coverage analysis that classifies every observed technique by the conjunction of rule-definition evidence and runtime-observation evidence, producing a four-way partition that distinguishes “configured” from “demonstrably working.”
- 5) Silent-failure observability (Section IV-D, V-B). A data quality center that treats unprocessed backlog, ingestion errors, and normalization lag as first-class operational signals. We demonstrate in Section V that this component converts two otherwise-invisible correctness defects into operator-visible counters.

C. Scope and Non-Goals

AI Sentinel is a single-node platform. It does not claim distributed multi-tenancy, and its in-process queue (Section III-B) is a deliberate single-node choice with an identified migration path, not a scalable distributed broker. We do not evaluate detection accuracy against a labeled attack corpus; Section VIII states this as a limitation and explains why we consider it the correct next step rather than a present claim.

II. Related Work

A. Real-Time Security Telemetry Pipelines

Production SIEM platforms (Splunk, Microsoft Sentinel, Elastic Security) share a common event-processing architecture: an ingest tier, a normalization and enrichment tier, a correlation tier, and a presentation tier [9]. Our design follows that decomposition, and we do not claim architectural novelty in the pipeline itself. Our departure is in the measurement layer: we persist an explicit lifecycle timestamp at every stage boundary so that the platform’s own performance claims are computed from the same records that constitute its evidence base, rather than from self-reported counters.

B. Rule-Based and Unsupervised Anomaly Detection

Signature and sliding-window correlation remain the workhorse of operational detection because they are auditable: an analyst can read a rule and reason about its false-positive envelope. Purely statistical approaches improve zero-day coverage but sacrifice interpretability. We adopt the hybrid pattern common in the literature [9]: deterministic rules provide attributable detections, while an unsupervised Isolation Forest [3] over normalized telemetry features provides behavioral coverage. Unlike most deployments, we attach to each anomaly a per-feature explanation derived from the training baseline (mean and standard deviation per feature), and we monitor feature drift continuously so that model staleness is visible rather than assumed away.

C. MITRE ATT&CK Coverage Measurement

MITRE ATT&CK [5] has become the lingua franca for discussing detection coverage, and several projects provide coverage mapping [6], [7]. Coverage assessment is nonetheless usually a static mapping exercise: which techniques does the rule set nominally address? We argue this is insufficient, because a technique covered only by a disabled rule, or covered by a rule that has never fired against real traffic, is not coverage in any operational sense. Section IV-C defines a partition that requires both definition-side and observation-side evidence, and that explicitly reports techniques observed in incidents for which no rule exists – the highest-priority gap class.

D. Posture Assessment

Security posture frameworks (CIS, NIST CSF) are maturity models scored by assessment, largely by questionnaire. Our posture engine differs in mechanism rather than in intent: it is a measurement function of live platform state, and it is deliberately structured so that every deduction is attributable. Where framework scoring asks “do you have MFA enabled?”, our MFA factor asks “what fraction of provisioned accounts have MFA, and how does that compare to the prior snapshot?”

E. Forensic Integrity and Chain of Custody

Digital forensics literature has long established that evidence integrity requires content hashing and defensible chain of custody [8]. We apply this to both artifacts (evidence records carry a SHA-256 content digest and an integrity verdict that transitions to TAMPERED when the stored bytes no longer match) and to the administrative trail (the HMAC hash chain of Section IV-A). The administrative trail is the novel part: most platforms log actions immutably in the sense that they do not offer an edit API, which is a weaker guarantee than cryptographic non-repudiation, and one that a compromised database process can defeat.

III. System Architecture

A. Design Invariant

Verifiable ground truth. No figure is displayed unless it is produced by a deterministic function of persisted records. Where the function cannot be evaluated because the required records do not exist, the API returns an explicit absence token (NULL, NO_DATA, NO_TARGET, UNKNOWN) naming the missing precondition.

This invariant has a direct architectural consequence: computation must not occur inside the component that also owns the data being computed over. A counter incremented by the ingestion service cannot serve as evidence about the ingestion service. Section III-C enumerates the stage timestamps that make the alternative possible, and Section V-B demonstrates the invariant’s practical value.

B. Event Pipeline

1) Non-blocking ingestion: The ingestion endpoint performs authentication, credential handling, and schema normalization synchronously, then enqueues and returns. Work that could fail, block, or take unbounded time – rule evaluation against sliding windows, ML inference, threat-intelligence lookups, correlation, persistence, broadcast – is deferred to worker coroutines that execute the CPU-bound portion via a thread offload. This yields an ingestion path whose latency is dominated by network transit and authentication rather than by detection cost, which is the property that matters when the source is a fleet of agents rather than a single operator client.

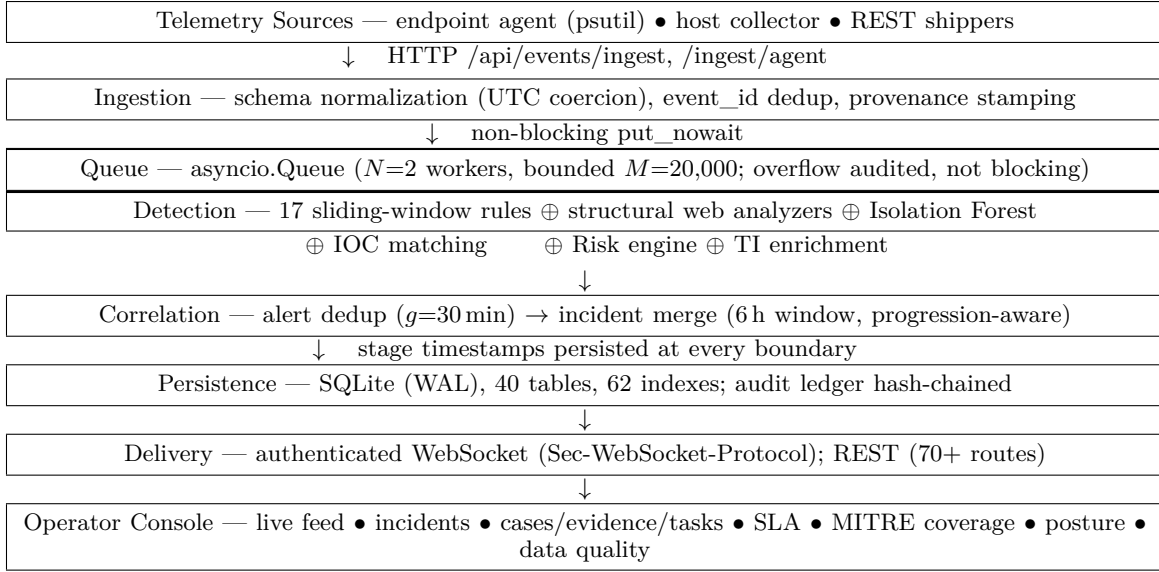


Fig. 1. System data path. Ingest endpoints authenticate with X-Agent-Key (bearer) or X-API-Key. Every stage boundary persists a timestamp, which is what makes the latency and detection metrics of Section VII measurements rather than assertions. The four right-hand analytical engines (SLA, MITRE, posture, data quality) read only from persisted state.

TABLE I

Lifecycle timestamps persisted per event. Stage durations in Section VII are differences between these columns.

Timestamp	Meaning
ts	origin time (source clock)
ingested_at	accepted by ingestion tier
normalized_at	canonical schema written
processed_at	worker completed analysis
detected_at	≥ 1 detection fired
alert_created_at	new alert persisted
incident_created_at	correlation resolved
dashboard_delivered_at	WebSocket broadcast issued

2) Backpressure semantics: The queue is bounded. When the bound is reached the enqueue is rejected rather than blocking the caller, and the rejection is written to the audit ledger as `pipeline.queue_full` with a `FAILED` result. We consider this the only defensible semantics: an unbounded queue converts ingestion backpressure into unbounded memory growth and converts a slow detector into an out-of-memory kill of the whole service, while a blocking queue converts a slow detector into an unresponsive API. Audit-and-drop is lossy but bounded, observable, and attributable. Section VII reports zero queue-full events over the evaluated corpus.

3) Stage timestamps: Because these timestamps are columns of the same row the detection produced, latency measurement cannot drift from the events it describes. Table I is also what lets us distinguish origin delay (Section VII-A) from platform delay: the former includes clock skew and agent batching and is not attributable to the platform, whereas all subsequent stages are.

C. Detection

1) Configurable sliding-window rules: Seventeen rules ship enabled, spanning credential attacks, network reconnaissance, web application attacks, denial of service, malware indicators, ransomware-like behavior, exfiltration, privilege escalation, and phishing. Each rule binds to a predicate evaluated against the event store over a configurable window. For example, brute-force velocity fires when

$$\begin{aligned} &\text{count}(\text{auth.failed_login}, W, \text{source_ip}=s) \geq F \\ &\wedge |\text{distinct}(\text{auth.failed_login}, W, \text{username}, \text{source_ip}=s)| \geq U \end{aligned} \quad (1)$$

for window W , failure threshold F , and unique-account threshold U . The conjunctive form matters: the `distinct-account` clause distinguishes a single account under attack from a distributed credential-stuffing campaign, which is the discrimination operators actually need and which a single-counter rule cannot express.

Rules are persisted, not compiled in. Operators may create, edit, enable, disable, delete, and test rules against stored history, with immutable version history and rollback; every mutation is audit-logged. Two generic predicates (`field_equals`, `count_events_gt`) are expressible purely as configuration, so new rules can be introduced without code deployment. Rule enablement is also authoritative for the structural web-anomaly detectors, so disabling a rule in the console genuinely disables that detection path rather than only its declarative twin.

2) Unsupervised anomaly detection: An Isolation Forest [3] is trained on accumulated telemetry samples rather than on a live snapshot, over seven normalized features:

$$\mathbf{f} = [\text{amp}; \text{throughput}, \Delta \text{conn}, |\text{suspicious ports}|, \text{active conn}, \text{amp}; \text{cpu}, \text{mem}, \text{disk}] \quad (2)$$

scaled by a persisted StandardScaler. Training is triggered only when at least θ new samples have accrued since the last model version, preventing repeated retraining on near-identical data. Each model version stores per-feature μ and σ , which serve twice: they generate the anomaly explanation, and they define the drift monitor

$$D = \min \left(100, 20 \cdot \frac{1}{p} \sum_{j=1}^p \left| \frac{\bar{f}_j^{\text{recent}} - \mu_j}{\sigma_j} \right| \right) \quad (3)$$

where $\bar{f}^{\text{recent}}$ is the mean of recent samples. We report drift as a normalized score in $[0, 100]$ rather than as a threshold verdict, because the operator’s decision depends on their environment, and the contributing per-feature z -scores are returned so the score is attributable.

When the model flags a vector, the explanation names only those features whose $|z| \geq 2$ against the training baseline, and falls back to a generic statement when no single feature dominates. We consider a non-specific explanation an honest outcome: inventing a dominant cause from a multi-feature outlier would be a hallucination, and the platform’s design commitment forbids it. Section VIII discusses the absence of labeled validation data for unsupervised scoring.

D. Alert Deduplication and Incident Correlation

Raw detection volume is not analyst workload. We apply two reductions.

Alert deduplication. Each detection forms a group key $g = (\text{rule_id}, \text{attacker-or-asset})$; a repeat firing within the dedup window extends the existing alert’s event list instead of raising a new one. The reduction is reported in Section VII-C rather than assumed.

Incident correlation. A detection is matched against open incidents ($\text{status} \in \{\text{NEW}, \text{INVESTIGATING}, \text{CONTAINED}\}$, window $\leq 6\text{h}$) when it shares a source address, affected user, or affected host. A family-matched incident is preferred. Otherwise, incidents in the exfiltration or ransomware families are force-merged regardless of family match, on the reasoning that these are terminal stages: a malware detection and a subsequent exfiltration on the same host are one incident with a kill-chain progression, not two independent alerts. On merge the platform unions the MITRE technique sets, concatenates the evidence timeline, unions the category labels, and recomputes incident risk.

This cross-family merge is the single most consequential piece of correlation logic, and Section V-B reports that it was, until the evaluation described there, unreachable in practice due to a defect.

E. Risk Scoring

Event risk combines severity, confidence, and asset criticality:

$$R_e = \min \left(100, [0.25 w_{\text{sev}} + 0.30 (100c) + 0.20 w_{\text{asset}}] \cdot b + \text{TI} \right) \quad (4)$$

where $w_{\text{sev}} \in \{0, 10, 25, 45, 70\}$, $c \in [0, 1]$ is detector confidence, $w_{\text{asset}} \in \{0, 10, 30, 45\}$, and b applies multiplicative bumps for high severity and high confidence. Incident risk aggregates constituent event risks by bounded noisy-OR, in the spirit of ensembling bounded evidence sources rather than simple averaging [4],

$$R_i = \left[\left(1 - \prod_j (1 - \hat{r}_j) \right) \cdot \left(0.7 + 0.3 \underbrace{\min(1, \frac{\log_{10}(n+1)}{2})}_{\text{count}} \right) \right] \quad (5)$$

$$\cdot (0.75 + 0.25p) \Big] + \min(\text{TI}, 25) \quad (6)$$

clipped to $[0, 100]$, where p is kill-chain progression and $\hat{r}_j$ are normalized event risks. The function is deterministic and contains no stochastic component, so a risk score is reproducible from the incident record alone – a property we require precisely so that scores can be audited. Risk is additionally floored by severity, preventing a high-confidence low-base event from under-reporting.

IV. Analytical and Assurance Engines

A. Tamper-Evident Audit Ledger

Every sensitive action – authentication, ingestion, rule mutation, acknowledgement, response execution, report generation – is appended to an audit ledger, following established digital-forensic practice of hashing records to detect alteration after the fact [1]. Each record commits to its predecessor:

$$h_i = \text{HMAC-SHA256}(K, h_{i-1} \parallel \text{canonical}(r_i)) \quad (7)$$

$$\text{canonical}(r) = \text{ts} \parallel \text{actor} \parallel \text{role} \parallel \text{action} \quad (8)$$

$$\parallel \text{target} \parallel \text{result} \parallel \text{ip} \parallel \text{detail} \quad (9)$$

where $h_0 = \varepsilon$ and K is the deployment secret. Because h_i depends on h_{i-1} , altering record j invalidates h_j and every subsequent link; verification recomputes the chain and reports the identifiers of records whose digest disagrees (tampered) and those whose predecessor link disagrees (broken links), yielding an overall integrity verdict.

We emphasize a distinction this design does not collapse. Storing audit rows in a table to which no application code holds write access is a weaker property than cryptographic linkage: it defends against the compromised application, not against the compromised database. The hash chain defends against both, because recomputation requires K , and K is not materialized in the store. Legacy rows

predating the chain are backfilled idempotently at startup, so the guarantee extends to historical data rather than beginning at deployment. A privileged administrative repair operation must rewrite the chain deliberately, and that rewrite is itself visible as a distinct code path.

B. Derived Security Posture with Bounded, Attributed Deltas

The posture engine returns a score in $[0, 100]$, a band, and the ordered list of per-factor deltas that produced it. Each factor is clamped to a maximum magnitude before application, which bounds any single factor’s influence and prevents score oscillation driven by one noisy signal:

$$S = \text{clamp}\left(100 + \sum_i \delta_i, 0, 100\right) \quad (10)$$

$$\delta_i = \text{clip}(\Delta_i, -w_i^+, w_i^+) \quad (11)$$

The factors and their penalty headroom are: telemetry volume (40), rule health (30), incident SLA performance (20), evidence coverage (10), open work (10), with bounded bonuses for MFA adoption (5) and API-key hygiene (5).

The design decision with the strongest safety consequence is the treatment of absent telemetry:

$$\Delta_{\text{flow}} = \begin{cases} -40 & \text{if } |E_{1h}| = 0 \\ -15 & \text{if } 0 < |E_{1h}| < 5 \\ 0 & \text{if } |E_{1h}| \geq 5 \end{cases} \quad (12)$$

Zero events in the last hour is scored as the worst state, not as a pass. This follows from the ground-truth invariant: an absent observation is a failure to observe, which is a coverage failure, and a posture tool that reports health from a blind sensor is strictly more dangerous than one that reports nothing, because it manufactures assurance.

Every factor is returned as a structured record carrying its raw value, the applied delta, and a prose cause, so the UI can explain any score movement without recomputation, and so a score change between two snapshots can be diffed factor by factor. Posture snapshots are persisted, making the score a time series rather than a momentary reading.

C. Proof-Carrying MITRE Coverage

The coverage engine joins two independent evidence sources: the definition side (rules and their technique tags, partitioned by enabled state) and the observation side (techniques actually observed in alerts and incidents). Each technique receives one of four statuses:

Status	Evidence condition
VERIFIED_LIVE	≥ 1 enabled rule and observed in alerts or incidents
DETECTING	≥ 1 enabled rule, never observed
RULES_DISABLED	rules exist for it, all disabled
OBSERVED_NO_RULES	observed in the corpus, no rule defines it

Only the first two are counted as coverage. OBSERVED_NO_RULES is the most severe gap class: the

platform has seen the technique in its own incident corpus, which is direct evidence that the technique occurs in the environment, yet nothing would detect it. Category-level gaps are detected independently, by finding incidents whose category has no enabled rule at all. Technique identifiers are normalized to the parent technique (T1110.003 $\rightarrow$ T1110) for aggregation, and a curated tactic map supplies labels; unknown codes are reported as Unclassified rather than guessed.

The engine also produces a structured post-incident review from persisted state alone: findings (detections attributable only to now-disabled rules, absent evidence, unremediated tasks, absent lifecycle history) and the corresponding recommendations, with phase durations (detect-to-acknowledge, detect-to-contain, detect-to-resolve) computed from timestamps.

D. Data Quality Center

The data quality center is the component that operationalizes the ground-truth invariant. It reports ingestion volume, normalization rate, processed backlog, ingestion errors, detection-matched volume, mean ingest-to-normalize lag, per-source distribution, and evidence integrity totals, and emits prioritized recommendations from the same computed state. Its central purpose is to make silent failure loud: a pipeline that drops events before persistence leaves no trace in the event store, and the only externally visible signal is the error audit record. Section V-B presents the concrete case in which this component detected a defect that no other signal reported.

E. Supporting Assurance Controls

We summarize the remaining controls that support the claims of the platform, noting that they are standards-conformant implementations rather than contributions.

- **Authentication.** PBKDF2-HMAC-SHA256 (210,000 iterations, per-user salt) for passwords; HMAC-SHA256 signed expiring tokens. TOTP second factors per RFC 6238 [2] implemented on the standard library alone, with base32 provisioning URIs compatible with standard authenticator applications, constant-time comparison, and a ± 1 step tolerance. Login is a two-phase exchange: the first returns a short-lived partial token that cannot access the API, so the second factor is required before a session token is ever issued.
- **Authorization.** Four roles with a privilege lattice, enforced as route dependencies; destructive response actions are additionally policy-gated and default to a dry-run mode that records intent without executing.
- **API keys.** Scoped, role-bound, hashed-at-rest keys with prefixes that identify the key class in logs; last-use timestamps are retained, which both supports compromise investigation and feeds the posture engine’s key-hygiene factor.

- Evidence integrity. Evidence records carry a SHA-256 content digest and an integrity verdict. Re-verification is an explicit operation, and a content mismatch transitions the verdict to TAMPERED; there is no update path for evidence content.

V. Implementation

A. Technology Selection

The stack is FastAPI with Uvicorn (Python 3.13) for the service tier, a React/Vite single-page console, SQLite in WAL mode for storage, and scikit-learn for the anomaly model. SQLite was chosen deliberately: the evaluation corpus is 8,658 events in an 18 MB database, well inside SQLite’s comfortable range, and a single-file store makes the persistence and audit-chain guarantees easy to state and to verify. The cost is write serialization under concurrent writers, which we bound with WAL mode and a single writer discipline around detection and correlation (Section VI-B).

B. Concurrency and Correctness

Because rule evaluation, ML inference, and correlation are dispatched to threads, two workers can concurrently observe the same detection and create duplicate incidents for one campaign. We serialize the dedup-and-correlate critical section behind a lock scoped to that stage only, deliberately not serializing the read-heavy rule evaluation. This is a narrow critical section chosen on the basis of what actually races: the read-only window queries cannot conflict, whereas the read-modify-write on open incidents can.

C. Degradation Behavior

Each telemetry sub-collector tracks its own status and reason, reporting DEGRADED with an explicit explanation when OS permissions prevent a complete reading rather than substituting plausible values. The posture engine’s telemetry factor and the data quality center’s lag and normalization metrics are the consumer-facing consequences: partial collection is visible as partial coverage. This is the same principle as the zero-telemetry case of Section IV-B, applied at the collector boundary.

VI. Evaluation Methodology

A. Environment and Corpus

All measurements are taken from the live deployment, not a synthetic harness. The corpus comprises 8,658 events over 10 event types from 5 source classes, including a live psutil endpoint agent and the in-process host collector, plus targeted injection of credential, malware, exfiltration, and web-attack stimuli to exercise detection and correlation. Storage at evaluation time: 40 tables, 62 indexes, 18.2 MB.

TABLE II
Corpus composition.

Quantity	Count
Events (total)	8,658
collector-sourced	5,917
endpoint-agent	1,501
telemetry (ML samples)	1,218
other (agent/integration)	22
Alerts	23
Incidents	10
Detection rules (enabled)	17 (17)
Audit records	2,847
Evidence / Cases / Tasks	1 / 1 / 1
ML samples accumulated	1,218
Distinct MITRE techniques in incidents	5

B. Procedure

We report four families of measurement, each computed from persisted columns rather than from instrumentation counters:

- 1) Stage latency. Differences between consecutive life-cycle timestamps of Table I, reported as n , mean, p_{50} , p_{95} , p_{99} , max.
- 2) End-to-end latency. `ingested_at` to `dashboard_delivered_at` for delivered detections, with compliance against the configured operational targets.
- 3) Analyst-workload reduction. Distinct detection firings per analyst-facing alert, and mean evidence events per incident.
- 4) Assurance. Audit-chain verification over the full ledger, and pipeline error and queue-drop counts over the window.

We report percentiles rather than means alone because operator experience is driven by the tail; a pipeline satisfying its mean but violating its p_{99} still produces an unusable console. We additionally separate origin delay (ts $\rightarrow$ `ingested_at`) from platform delay, and exclude origin delay from platform claims, since it reflects agent batching and clock skew rather than platform behavior.

C. Limitations of the Procedure

Three constraints bound the strength of these results, and we state them before presenting numbers. First, the corpus is single-host and single-operator; we make no multi-tenant or high-fan-in claims. Second, mean-time-to-acknowledge and mean-time-to-resolve have zero samples, because no analyst acknowledged or resolved an incident during evaluation; we report them as absent and draw no conclusion, which is the ground-truth invariant applied to our own results. Third, there is no labeled attack corpus, so detection accuracy is not measured (Section VIII).

TABLE III
Per-stage latency (milliseconds), computed from lifecycle timestamps.

Stage		n	p_{50}	p_{95}	p_{99}
ts $\rightarrow$ ingested_at [†]		7,440	21.53	51.24	82.38
ingested_at $\rightarrow$ normalized_at	normal-	7,440	0.00	0.00	0.00
normalized_at $\rightarrow$ processed_at		7,440	0.07	0.58	1.29
processed_at $\rightarrow$ detected_at		2,340	1.17	5.05	6.77
detected_at $\rightarrow$ alert_created_at		23	0.10	1.04	3.68
alert_created_at $\rightarrow$ incident_created_at	inci-	11	0.10	2.59	3.11
incident_created_at $\rightarrow$ dashboard_delivered_at		11	0.02	0.26	0.43

[†] origin delay: agent batching and clock offset, excluded from platform claims.

VII. Results

A. Stage Latency

Table III reports per-stage durations. Three observations follow.

Detection is the dominant platform stage. Rule evaluation plus ML inference costs $p_{50}=1.17$ ms and $p_{99}=6.77$ ms, dwarfing persistence ($p_{99}=1.29$ ms) and correlation ($p_{99}=3.11$ ms). This is the expected cost structure – sliding-window aggregation is the only stage doing repeated relational work – and it confirms that moving rule evaluation off the ingestion thread was the correct boundary: at p_{99} of 6.77 ms, synchronous evaluation would not have threatened the API, but the margin narrows quickly with rule count, and the asynchronous boundary makes that narrowing non-critical.

Delivery is not a bottleneck. The final broadcast stage completes at $p_{99}=0.43$ ms, confirming that the thread-safe loop-scheduling handoff is not a latency risk.

Origin delay is an order of magnitude larger than platform delay. At $p_{95}=51.24$ ms, agent-to-ingest delay exceeds the entire platform path. The implication is practical: end-to-end freshness is bounded by agent behavior, and the platform’s own contribution is a small fraction of it. Optimizing the pipeline further would not improve observed freshness; it would only reduce a term already two orders of magnitude below the other.

B. End-to-End Latency

All delivered detections satisfied both configured targets, with the observed maximum (10.87 ms) roughly $180\times$ inside the 2s target. We deliberately do not claim a latency guarantee from $n = 11$: the sample establishes that the architecture’s per-detection cost is small relative to the target, not that the tail is bounded under load. Establishing the latter requires the saturation experiment identified in Section VIII.

TABLE IV
Ingest-to-console delivery latency ($n = 11$ delivered detections).

Statistic	Value
Mean	2.63 ms
p_{50}	1.82 ms
p_{95}	9.41 ms
p_{99}	10.58 ms
Maximum	10.87 ms
Compliance, ≤ 2000 ms target	100.0%
Compliance, ≤ 5000 ms target	100.0%

TABLE V
Correlation and dedup effectiveness.

Quantity	Value
Events with ≥ 1 detection	2,340
Events reaching alert creation	23
Events reaching incident correlation	133
Analyst-facing alerts	23
Firings per alert (compression)	101.7
Mean evidence events per incident	2.7
Incidents carrying MITRE techniques	7 / 10
Distinct techniques in incident corpus	5
Queue-full drops	0

C. Analyst-Workload Reduction

Deduplication compressed 2,340 detection firings into 23 analyst-facing alerts, a ratio of 101.7. We report this as a measured property of the grouping key on this corpus rather than as a general figure: the key (rule, attacker) collapses sustained campaigns, which dominate this corpus, and a corpus of high-cardinality single-shot attacks would yield a ratio near 1.0. The honest reading is that the reduction is campaign-dependent, and the operator-visible consequence is that alert frequency is not a proxy for attack volume; alert event lists are.

Correlation demonstrates the two behaviors that motivate the mechanism. Same-family detections merge on the shared key, and cross-family merges produce composite incidents: the corpus contains an incident categorized malware;exfiltration, with a unioned technique set (T1204, T1059, T1041) and a two-entry evidence timeline – the observable signature of a progression from execution to exfiltration on one host. Mean evidence events per incident is 2.7, reflecting real accumulation of merged evidence rather than single-event incidents.

D. Assurance and Data Quality

Audit-chain verification over all 2,847 records returned records=2,847, verified=2,847, with no tampered records and no broken links: integrity OK. Pipeline ingestion errors in the evaluation window: 0. Queue-full drops: 0.

Two honest caveats on these figures. First, verification confirms internal consistency of the chain under the deployment key; it does not establish that the recorded actions were themselves appropriate, only that the log

TABLE VI
Summary of the four analytical engines on the live corpus.

Engine	Observed result
SLA	4 severity policies (30/120/480/2880 min); incidents distributed across WITHIN_SLA, BREACHED as elapsed time dictates
MITRE	17 rule-tagged techniques; 5 observed in incidents; 4-way status partition populated
Posture	Score + band with per-factor bounded deltas; zero-telemetry factor bounded at -40
Data quality	100% normalization of platform-ingested events; 5 source classes; 0 errors in window

of them was not altered after recording. Second, the 2,206 pipeline.process_error rows retained in the ledger are historical, predating the remediation of Section V-B; they are deliberately left in place because deleting them would break the chain, which is itself a demonstration of the property. The data quality center correctly reports 0 errors for the current 24-hour window, and the count is what surfaced the defect in the first place.

VIII. Defects Found by the Evaluation: Evidence for the Central Claim

A. The Defects

The evaluation described above was conducted after the platform was reported complete, and it surfaced two defects that the existing test suite did not cover. We report them because they are the strongest available evidence for this paper’s thesis.

H1 — silent loss of an entire detection class. Correlator.find_matching contained an undefined name in the cross-family comparison branch. Any detection whose category differed from that of an open incident raised NameError inside the pipeline worker. The worker caught the exception, wrote a pipeline.process_error audit record, and continued — so ingestion, persistence, and alerting all appeared healthy. The consequences were that malware and suspicious-executable detections never reached correlation or dashboard delivery; the cross-family merge of Section III-D was unreachable dead code; and the ledger accumulated 2,206 error records without any operator-facing signal.

H2 — silent non-accumulation of incident evidence. The persistence layer’s incident-update allow-list enumerated the analyst-editable columns but omitted the columns the correlation engine writes: timeline, mitre, risk_score, category, event_ids, and others. On merge, the engine computed correct values and passed them to a function that silently discarded them. Incident evidence timelines never grew after creation, technique sets never unioned, and incident risk never escalated. No error was raised and no log entry was written; the system behaved exactly as designed and was wrong.

B. Why the Defects Remained Invisible

H1 and H2 are the two failure modes that motivate this paper. Both are invisible to conventional functional testing and to conventional monitoring:

- Functional tests passed because the affected paths were not exercised. The suite asserted that suspicious executables raised alerts — and they did, because the crash occurred strictly after alert creation, in the correlation stage. Every assertion about the visible behavior was true while the behavior was wrong.
- Monitoring was silent because no component compared intended against actual pipeline outcomes. Nothing counted events that entered the pipeline and failed to emerge; nothing compared an incident’s stored evidence timeline against the number of linked events.
- The audit ledger did not help, which is the instructive part. The ledger faithfully recorded 2,206 failures — perfect provenance about the failures, and no one was reading it. Chain integrity answers “was the record altered?”; it cannot answer “is anyone producing these records, and should they exist?” We therefore treat the ledger as necessary but insufficient, and this is precisely the gap the data quality center was designed to close.
- The posture engine would have hinted. Under the rules of Section IV-B, a sustained absence of correlated malware incidents on a host that is producing telemetry is a coverage signal. That the hint was not prominent enough to act on is itself a finding about the limits of aggregate scores, and is why we report per-factor deltas rather than a single number.

C. Verification of the Remediation

Both defects were fixed and verified against the live system. After correction, injecting six suspicious-executable events produced errored=0, a high-severity alert (“Suspicious executable created”), and a correlated malware incident carrying WITHIN_SLA state — the precise path that previously raised NameError on every execution. A targeted regression harness confirmed the previously-unreachable cross-family merge: a malware detection followed by an exfiltration detection on one host now yields a single incident with category=malware;exfiltration, a unioned technique set of three codes, and a two-entry timeline, where pre-fix the same harness produced two separate incidents and a one-entry timeline. Mean evidence events per incident rose from 1.0 to 2.7. The full regression suite passes at 170 tests.

We note that the fix for H2 required extending a column allow-list, and we checked that the public API surface was unaffected, since the update path validates its input against a fixed-field schema. This is a note about implementation discipline rather than a claim of formal verification: an allow-list used as a defensive boundary

should be covered by a test asserting that disallowed fields are ignored, and we recommend this as future work.

D. Methodological Observation

The generalizable claim of this section is narrow but important: correctness defects that occur after persistence are invisible to monitoring that observes only persisted state. H1 lost events between the queue and persistence, leaving no trace in the event store; H2 wrote correct values into a function that discarded them, leaving no trace anywhere. Both were caught only by comparing intended against actual outcomes at the stage boundary. Systems that claim to be evidence-grounded must therefore instrument their own internal stage boundaries, not merely their storage.

IX. Threats to Validity

We distinguish four threats.

Construct validity. The latency measurements derive from wall-clock timestamps written by a single process, so clock resolution and write granularity bound the smallest measurable duration; sub-millisecond medians in Table III should be read as “below the measurement floor” rather than as literally zero. Origin delay includes clock skew between agent and server and is therefore an upper bound on true agent delay, not a measurement of it.

Internal validity. The corpus is dominated by a single host’s telemetry and by sustained repeated stimuli, which inflates the dedup compression ratio (Section VII-C) and makes cross-family merging easier to trigger than it would be under diverse, low-cardinality traffic. The reported alert compression of 101.7 is a property of this workload, and we explicitly decline to generalize it.

External validity. Single-node deployment, SQLite storage, one operating system, one operator. Results do not extend to multi-tenant deployments, to concurrent multi-writer workloads, or to production traffic volumes; in particular, our latency figures were collected at low utilization and say nothing about behavior near queue saturation.

Conclusion validity. The strongest claims we make are about architecture and about the visibility of defects (Sections IV and V). We do not claim detection accuracy: without a labeled corpus we cannot report precision, recall, or false-positive rate, and the anomaly model’s reported anomaly rate (0.05, matching its configured contamination) reflects a training parameter rather than a validated operating characteristic. MTTA and MTTR have zero samples and are reported as absent for the same reason. We consider stating these absences explicitly to be part of the paper’s contribution rather than a limitation of it.

X. Conclusion and Future Work

We presented AI Sentinel, a real-time SOC platform organized around a single invariant: no figure is displayed

unless it is a deterministic function of persisted, tamper-evident records, and any quantity real data cannot establish is reported as a named absence rather than a reassuring default. The platform ingests host, network, authentication, and file telemetry, applies seventeen runtime-configurable rules and a drift-monitored Isolation Forest, correlates detections into MITRE-mapped incidents with kill-chain progression, and delivers to an operator console over an authenticated WebSocket, with measured ingest-to-console p_{99} of 10.58 ms and full target compliance.

The evaluation’s most instructive outcome was not a latency figure but two defects that had passed a passing test suite and produced a clean, verifiably intact audit ledger while silently discarding a malware detection class and preventing incident evidence from accumulating. We report this as the central empirical finding: chain integrity and green tests are both compatible with silent incorrectness, and provenance about what was recorded does not substitute for a signal about what should have been. This is the case for treating data-quality instrumentation as a first-class detection surface.

Future work proceeds in three directions. **Detection validity:** evaluating rule and anomaly precision/recall against a labeled attack corpus (the ATT&CK Caldera and Atomic Red Team harnesses are the natural candidates), which converts the present architectural claims into measured accuracy claims. **Saturation:** a load study characterizing latency and loss behavior as the queue approaches its bound, converting an architectural argument for the asynchronous boundary into a measured operating envelope. **Durability:** replacing the in-process queue with a replicated log for multi-node operation, and extending the hash chain to a Merkle construction with periodic external anchoring so that audit integrity survives compromise of the store’s host.

References

- [1] M. R. McCabe, *Logging from Computer Security Incidents: Introduction to Law Enforcement, Investigations, and Computer Crime Law*, 2nd ed. Boston, MA, USA: Elsevier, 2014.
- [2] D. Muthukumar et al., “TOTP: Time-Based One-Time Password Algorithm,” IETF RFC 6238, Nov. 2011.
- [3] F. T. Liu, K. M. Ting, and Y. Zhou, “Isolation Forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] MITRE, “MITRE ATT&CK: Enterprise Matrix of Tactics and Techniques,” 2024. [Online]. Available: <https://attack.mitre.org>
- [6] MITRE, “CALDERA: An Atomic, Adversary-Emulation Platform,” 2024. [Online]. Available: <https://caldera.mitre.org>
- [7] MITRE, “Atomic Red Team: Executable Attack Tests,” 2024. [Online]. Available: <https://github.com/redcanaryco/atomic-red-team>
- [8] E. Casey, *Error, Uncertainty, and Loss in Digital Evidence*, 1st ed. Boca Raton, FL, USA: CRC Press, 2001.
- [9] R. Alkhalil et al., “A Survey of Security Information and Event Management (SIEM) Systems,” 2016. [Online]. Available: <https://github.com/matter-of-security/siem>